

Trading Station IA

Guide d'intégration : Screenshots TradingView vers PostgreSQL

Ce document détaille la procédure technique pour capturer des images depuis le widget TradingView et enregistrer leurs références dans une base de données PostgreSQL hébergée sur Render.

1. Schéma de la Base de Données

Exécutez cette requête SQL dans votre console Render Postgres pour créer la table nécessaire :

```
CREATE TABLE IF NOT EXISTS trading_capture (  
  id SERIAL PRIMARY KEY,  
  symbole VARCHAR(20),  
  intervalle VARCHAR(10),  
  image_url TEXT,  
  date_capture TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

2. Modifications : server.js (Backend)

Ajoutez ces routes à votre fichier existant pour gérer l'enregistrement et la récupération des captures.

```
// Route pour enregistrer une capture  
app.post("/api/save-screenshot", async (req, res) => {  
  const { symbole, intervalle, image_url } = req.body;  
  try {  
    const result = await pool.query(  
      "INSERT INTO trading_capture (symbole, intervalle, image_url)  
VALUES ($1, $2, $3) RETURNING *",  
      [symbole, intervalle, image_url]  
    );  
    res.json({ ok: true, data: result.rows[0] });  
  } catch (err) {  
    console.error("Erreur DB:", err);  
    res.status(500).json({ ok: false, message: "Erreur lors de la  
sauvegarde" });  
  }  
});
```

```
// Route pour récupérer l'historique
app.get("/api/captures", async (req, res) => {
  try {
    const result = await pool.query("SELECT * FROM trading_capture ORDER
    BY date_capture DESC");
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ ok: false });
  }
});
```

3. Modifications : index.html (Frontend)

Interface (HTML)

Ajoutez le bouton de capture dans votre barre d'outils :

```
<button onclick="capturerGraphique()" style="background: #00d9ff; color:
#000; font-weight: bold;">
 Capture Screenshot
</button>
<div id="liste-captures" style="margin-top: 20px;"></div>
```

Logique JavaScript

Utilisez la méthode `createScreenshot()` du widget `TradingView` :

```
async function capturerGraphique() {
  if (!widgetTradingView) return alert("Widget non chargé");

  widgetTradingView.activeChart().createScreenshot().then(async (url) => {
    // L'URL retournée est un identifiant unique (ex: abcdef)
    const imageLien = "https://www.tradingview.com/x/" + url;

    try {
      const reponse = await fetch(API_BASE_URL + "/api/save-
      screenshot", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          symbole: document.getElementById("symbol-input").value
          || "BTCUSDT",
          intervalle: "1h",
          image_url: imageLien
        })
      });
    } catch (err) {
      console.log(err);
    }
  });
}
```

```
        })
    });

    if (reponse.ok) {
        alert("Capture enregistrée avec succès !");
        chargerCaptures();
    }
} catch (erreur) {
    console.error("Erreur réseau:", erreur);
}
});
}
```

Note technique : Cette méthode ne stocke pas le fichier binaire (lourd) dans Postgres, mais l'URL permanente générée par TradingView. C'est la solution la plus optimisée pour les plans gratuits de Render.

Important : Assurez-vous que votre variable `API_BASE_URL` pointe bien vers votre service Render (<https://trading-g8ie.onrender.com>).